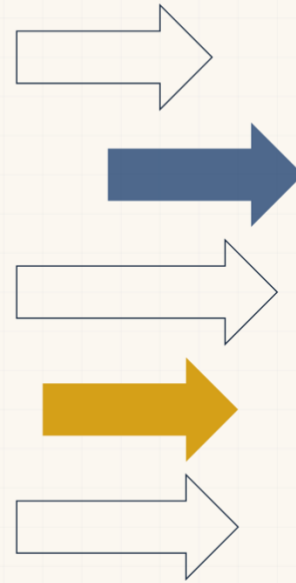


Programación I

Algoritmos, lógica, diagramas de flujo y pseudocódigo



90 / 60

AGENDA GENERAL · 15 HORAS

Qué vamos a aprender, en cinco sesiones

SESIÓN 01 · 3 H

Algoritmos y resolución de problemas

- › Qué es un algoritmo
- › Características: precisión, definición, finitud
- › Entrada · Proceso · Salida
- › Analogías cotidianas
- › Errores frecuentes del principiante

SESIÓN 02 · 3 H

Datos y diagramas de flujo

- › Identificadores
- › Variables y constantes
- › Tipos de datos básicos
- › Simbología de diagramas
- › Ejemplo: volumen de un cubo

SESIÓN 03 · 3 H

Pseudocódigo y estructuras secuenciales

- › Reglas y convenciones
- › De problema a pseudocódigo
- › Ejemplos paso a paso
- › Práctica guiada
- › Síntesis del módulo

SESIÓN 04 · 3 H

Estructuras selectivas (decisiones)

- › Selectiva simple (Si...)
- › Selectiva doble (Si...Sino)
- › Selectiva múltiple
- › Operadores lógicos
- › Ejemplos resueltos

SESIÓN 05 · 3 H

Estructuras repetitivas y arreglos

- › Mientras / Repite / Desde
- › Contadores y acumuladores
- › Ahorro, Fibonacci
- › Vectores y matrices
- › Integración final

Cómo vamos a aprender

«Programar no es escribir código. Programar es **resolver problemas paso a paso**».

01

Pensar antes de escribir

Primero entendemos el problema (qué entra, qué sale), luego trazamos los pasos. La computadora ejecuta — nosotros pensamos.

02

Ver, dibujar y escribir

Cada algoritmo lo representaremos de dos formas: **diagrama de flujo** (visual) y **pseudocódigo** (texto). Ambos se complementan.

03

Practicar en cada bloque

Después de cada concepto hay una **mini actividad** guiada. Los conceptos se asientan haciendo, no solo escuchando.

04

Equivocarse está permitido

Cada módulo incluye una sección de **errores frecuentes**. Equivocarse y corregir es la herramienta más poderosa para aprender a programar.

01

MÓDULO • SESIÓN 01 • 3 HORAS

Algoritmos y resolución de problemas

Entender qué es un algoritmo, cómo lo usamos todos los días sin darnos cuenta, y cómo se descompone cualquier problema en **entrada • proceso • salida**.

8 TEMAS • 1 ACTIVIDAD GUIADA • SÍNTESIS FINAL

Qué cubrimos

00:00 – 00:25	01 ¿Qué es un algoritmo? Definición y por qué importa
00:25 – 00:50	02 Analogía cotidiana La receta de cocina como algoritmo
00:50 – 01:20	03 Características esenciales Precisión · Definición · Finitud
– Pausa 10 min –	
01:30 – 02:10	04 Modelo Entrada · Proceso · Salida Cómo descomponer cualquier problema
02:10 – 02:40	05 Actividad guiada Diseñar 3 algoritmos sencillos en equipo
02:40 – 03:00	06 Errores frecuentes & síntesis Repaso y cierre del módulo

PROGRAMACIÓN 1 – MÓDULO 1 – AGENDA

OBJETIVOS

Al terminar serás capaz de...

- ✓ Definir qué es un algoritmo con tus propias palabras.
- ✓ Identificar las tres características esenciales.
- ✓ Aplicar el modelo E-P-S a cualquier problema cotidiano.
- ✓ Diseñar tu primer algoritmo paso a paso.

04 / 60

¿Qué es un algoritmo?

+ PREGUNTA DETONADORA

¿Has explicado alguna vez a alguien cómo llegar a tu casa? Si lo hiciste paso a paso... ya escribiste tu primer algoritmo.

DEFINICIÓN

Un **algoritmo** es una secuencia *ordenada, precisa y finita* de pasos para resolver un problema o realizar una tarea.

Lo que diferencia un algoritmo de unas "instrucciones cualquiera" es que cumple **tres condiciones** al mismo tiempo (las vemos en el siguiente tema).

¿POR QUÉ IMPORTA?

Es la base de TODO lo que aprenderás.

Cualquier programa — de la app del banco al buscador de Google — es, en el fondo, una colección de algoritmos.

Aprender a **pensar en algoritmos** es la habilidad central de la programación. El lenguaje (Python, Java...) viene después.

Una receta de cocina es un algoritmo

RECETA • HUEVO FRITO

Cómo preparar un huevo

1. Calienta la sartén a fuego medio durante 1 minuto.
2. Añade una cucharada de aceite.
3. Rompe el huevo en un plato pequeño.
4. Vierte el huevo con cuidado en la sartén.
5. Cocina 3 minutos hasta que la clara esté firme.
6. Sirve en un plato y agrega sal al gusto.

FIN Huevo frito listo.

Una receta cumple **exactamente** lo mismo que un algoritmo de computadora:

✓ PRECISIÓN

Cada paso es claro. No dice "cocina un poco", dice **3 minutos**.

✓ ORDEN

El paso 4 va después del 3. Si rompes el huevo antes de calentar, no funciona.

✓ FINITUD

Termina en algún momento. No cocinas "para siempre": a los 3 minutos paras.

Las tres reglas de oro

Cualquier conjunto de pasos que NO cumpla las tres condiciones siguientes **no** es un algoritmo válido.

CARACTERÍSTICA 01

Precisión

Cada paso debe indicarse de forma **clara y sin ambigüedad**. No puede haber dos interpretaciones posibles.

✓ BIEN

«Suma 5 a la variable A»

✗ MAL

«Súmame algo a A»

CARACTERÍSTICA 02

Definición

Si ejecutamos el algoritmo **muchas veces con los mismos datos de entrada**, debe producir **siempre el mismo resultado**.

✓ BIEN

$2 + 3 = 5$ (hoy, mañana, siempre)

✗ MAL

«Elige un número al azar» (varía cada vez)

CARACTERÍSTICA 03

Finitud

Debe **terminar en algún momento** con un número limitado de pasos. Un algoritmo que no termina, no sirve.

✓ BIEN

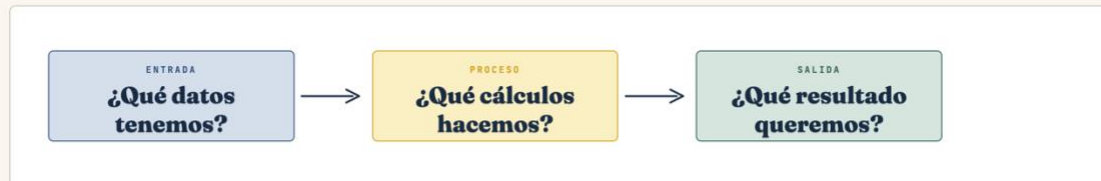
«Cuenta hasta 100 y termina»

✗ MAL

«Suma uno y vuelve al principio» (ciclo infinito)

Todo problema se descompone en tres

Antes de escribir un solo paso, hacemos tres preguntas. Si las contestas, el algoritmo casi se escribe solo.



EJEMPLO APLICADO · PROMEDIO DE 3 CALIFICACIONES

ENTRADA Tres calificaciones del estudiante: C1, C2, C3 Por ejemplo: 8.5, 7.0, 9.2	PROCESO Sumar las tres y dividir entre 3 $P = (C1 + C2 + C3) / 3$	SALIDA Promedio P del estudiante Por ejemplo: $P = 8.23$
---	--	--

PROGRAMACIÓN 1 – M1 · MODELO E·P·S

88 / 68

Vamos a aplicarlo a un problema real

PROBLEMA

Un cliente compra **tres productos** en una tienda. Queremos calcular el **total a pagar**, incluido el **16 % de IVA**.

PASO 1 · ANALIZAR EL PROBLEMA

ENTRADA Lo que el cliente nos da Producto 1 P1 Real Producto 2 P2 Real Producto 3 P3 Real Necesitamos 3 datos numéricos que el usuario nos va a teclear.	PROCESO Cómo lo calculamos 1. Calcular subtotal: $SUB = P1 + P2 + P3$ 2. Calcular IVA (16 %): $IVA = SUB \cdot 0.16$ 3. Calcular total: $TOT = SUB + IVA$	SALIDA Qué mostramos al usuario Subtotal SUB IVA IVA Total TOT Ejemplo: con $P1=100, P2=50, P3=80 \rightarrow$ $SUB=230, IVA=36.8, TOT = 266.8$
---	--	--

PROGRAMACIÓN 1 – M1 · EJEMPLO E·P·S

89 / 68

Tu turno: descompón en E · P · S

En parejas, identifica para cada problema sus tres componentes. No hay que escribir código todavía — solo **pensar**.

PROBLEMA 01 • FÁCIL

El área de un rectángulo

Conocidos el largo y el alto de un rectángulo, calcular su área.

- Ⓔ ¿Qué números necesitamos pedir?
- Ⓟ ¿Qué fórmula aplicamos?
- Ⓢ ¿Qué resultado mostramos?

PROBLEMA 02 • MEDIO

Edad en días

Dada la edad de una persona en años completos, calcular su edad aproximada en días.

- Ⓔ ¿Qué dato pide el algoritmo?
- Ⓟ ¿Por cuánto hay que multiplicar?
- Ⓢ ¿Cuál es la unidad de salida?

PROBLEMA 03 • RETO

Cambio en una compra

Un cliente paga un producto. Calcular cuánto cambio se le debe regresar.

- Ⓔ ¿Cuántos datos pedimos?
- Ⓟ ¿Resta o suma?
- Ⓢ ¿Qué mostramos?

✦ **COMPROBACIÓN** Compartan su análisis con otro equipo. ¿Coincide la entrada? Si no, comenten por qué cada uno escogió esos datos.

△ ERRORES FRECUENTES DEL PRINCIPIANTE

Lo que más se equivoca al empezar

Estos errores son **normales**. Identificarlos a tiempo te ahorrará horas de frustración.

01 Empezar a escribir antes de pensar

La tentación es lanzarse al pseudocódigo sin analizar. Resultado: te trabas a la mitad y borras todo.

Antídoto: Antes de cualquier paso, escribe en una hoja: *entrada, proceso, salida*.

02 Usar palabras imprecisas

«Súmale algo», «hazlo varias veces», «termina cuando esté listo». La computadora **no entiende** ambigüedad.

Antídoto: Si una abuela no podría seguir tu paso, vuelve a escribirlo.

03 Olvidar el paso «Fin»

Si tu algoritmo no tiene un punto donde se detiene, viola la **finitud**.

Antídoto: Todo algoritmo empieza con *Inicio* y termina con *Fin*. Es como una novela.

04 Confundir el lenguaje con el algoritmo

«¿Cómo se hace eso en Python?» No importa todavía. Primero el algoritmo, después el lenguaje.

Antídoto: Un buen algoritmo se traduce a CUALQUIER lenguaje. Por eso lo aprendemos primero.

Lo que nos llevamos

Cinco ideas que deben quedar muy claras antes de avanzar al Módulo 2:

- 1 Algoritmo = pasos ordenados para resolver un problema**
No es solo «programar», es *pensar* antes de programar.
- 2 Tres reglas: precisión, definición, finitud**
Si faltan, no es un algoritmo válido.
- 3 Modelo E • P • S es universal**
Aplica a cualquier problema, desde la calculadora hasta una nave espacial.
- 4 Primero analizamos; después escribimos**
El 80 % del trabajo está en entender el problema.
- 5 Un buen algoritmo es independiente del lenguaje**
Da igual Python, Java o C. Si el algoritmo es bueno, se traduce a todos.

PRÓXIMA SESIÓN

Datos y diagramas de flujo

Aprenderemos a darle nombre a la información (identificadores), clasificarla por tipo y representar nuestros algoritmos con diagramas visuales claros.

PROGRESO DEL CURSO

3 h / 15 h • Módulo 1 de 5 completado

02 Datos y diagramas de flujo

Aprender a darle **nombre** a la información, clasificarla por **tipo**, y representar nuestros algoritmos de forma **visual** con diagramas de flujo claros y limpios.

10 TEMAS • 1 ACTIVIDAD GUIADA • 1 EJEMPLO RESUELTO

Qué cubrimos en estas 3 horas

00:00 – 00:30	01 Identificadores Cómo nombramos a los datos
00:30 – 01:00	02 Variables y constantes Los dos tipos de identificadores
01:00 – 01:30	03 Tipos de datos básicos Entero · Real · Carácter · Lógico
– Pausa 10 min –	
01:40 – 02:10	04 Simbología de diagramas Los símbolos estándar
02:10 – 02:35	05 Operadores Aritméticos y de comparación
02:35 – 03:00	06 Ejemplo: volumen del cubo + actividad Primer diagrama de flujo completo

PROGRAMACIÓN 1 – M2 · AGENDA

OBJETIVOS

Al terminar serás capaz de...

- ✓ Nombrar correctamente variables y constantes.
- ✓ Elegir el tipo de dato adecuado.
- ✓ Reconocer los símbolos del diagrama de flujo.
- ✓ Dibujar tu primer diagrama de flujo completo.

14 / 60

¿Cómo le ponemos nombre a un dato?

PREGUNTA DETONADORA

Si en un problema necesitas tres números diferentes para calcular un promedio... ¿cómo le dirías a la computadora «quiero ESE número, no este otro»?

DEFINICIÓN

Un **identificador** es el *nombre* que le damos a un dato dentro de un algoritmo para poder referirnos a él.

REGLAS PARA UN BUEN IDENTIFICADOR

- Empieza por una **letra** (no por un número).
- Es **descriptivo**: usa la letra inicial de lo que representa.
- **No usa espacios** ni acentos ni caracteres especiales.
- Es **único**: dos variables distintas no pueden tener el mismo nombre.

EJEMPLOS

```
// edad de una persona
edad

// área de un rectángulo
area

// promedio de 3 notas
P o promedio

// valor constante PI
PI
```

PROGRAMACIÓN 1 – M2 · IDENTIFICADORES

15 / 60

Variables vs. constantes

La diferencia es sencilla: ¿el valor **cambia** mientras corre el algoritmo, o **no cambia**?

IDENTIFICADOR TIPO Variable

Su valor **puede cambiar** durante la ejecución. Es como una caja con etiqueta: el contenido cambia, la etiqueta no.

EJEMPLO

```
// al inicio:
edad = 5
// más adelante:
edad = edad + 1 // edad vale 6
```

📌 La misma variable `edad` tomó dos valores distintos en momentos distintos.

IDENTIFICADOR TIPO Constante

Su valor **no cambia jamás** durante la ejecución. Se asigna una vez y se respeta.

EJEMPLO

```
// declaración:
PI = 3.1416
IVA = 0.16
// se usan, pero NO se modifican
```

📌 Si en algún paso intentas hacer `PI = 4`, es un error: las constantes son intocables.

Cada variable guarda un *tipo* de dato

Antes de usar una variable, hay que saber **qué clase de información va a guardar**: ¿un número entero? ¿uno con decimales? ¿una letra? ¿sí/no?

TIPO 01

Entero

int • integer

Números **sin decimales**, positivos, negativos o cero.

EJEMPLOS

```
7    -3    0
1000 -250
```

Úsalo para:

edad, número de hijos, año, cantidad de productos

TIPO 02

Real

float • double

Números **con decimales**. Se separan con punto, no con coma.

EJEMPLOS

```
3.14  -0.5
9.81  100.0
```

Úsalo para:

precio, peso, temperatura, promedio, porcentajes

TIPO 03

Carácter

char • string

Letras, palabras o textos. Se escriben entre comillas.

EJEMPLOS

```
'A'  'z'
"María"  "Hola mundo"
```

Úsalo para:

nombre, apellido, sexo (M/F), dirección, código

TIPO 04

Lógico

boolean • bool

Solo dos valores posibles: **verdadero** o **falso**.

EJEMPLOS

```
verdadero
falso
true / false
```

Úsalo para:

aprobó/reprobó, está activo, es mayor de edad

La tabla de variables

Antes de escribir el algoritmo, hacemos una **tabla con todas las variables** que vamos a usar. Es como armar el inventario antes de cocinar.

PROBLEMA EJEMPLO

Calcular el sueldo semanal de un empleado conociendo sus horas trabajadas y el pago por hora.

NOMBRE	DESCRIPCIÓN	TIPO	USO
HT	Horas trabajadas en la semana	ENTERO	Entrada
PH	Pago por hora trabajada	REAL	Entrada
SUE	Sueldo total calculado	REAL	Salida
NOM	Nombre del empleado	CARÁCTER	Entrada

TIP PEDAGÓGICO

No saltes este paso, nunca.

Cuando aprendes, la tabla de variables parece "trabajo extra". Pero te **obliga a pensar** qué datos necesitas y de qué tipo.

Cuando lleves 50 algoritmos resueltos, podrás saltártela mentalmente. Hasta entonces, **siempre** hazla.

Dibujar un algoritmo, paso a paso

DEFINICIÓN

Un **diagrama de flujo** es la representación *gráfica* de un algoritmo, hecha con figuras estándar conectadas por flechas que indican el orden de ejecución.

¿PARA QUÉ NOS SIRVE?

- Comprender el algoritmo de un vistazo**
Una imagen vale más que mil líneas de pseudocódigo, especialmente cuando estás empezando.
- Detectar errores antes de programar**
Si la lógica está mal, lo ves dibujado. Te ahorra horas de depuración.
- Comunicar la idea a otra persona**
Es un lenguaje universal: cualquiera lo lee, sin saber programar.

VISTA PREVIA

Cómo se ve un diagrama mínimo:



Los siete símbolos que debes conocer

Cada figura tiene un significado fijo. Memorízalas: las vas a usar siempre.



Terminal · Inicio/Fin
Marca dónde empieza y dónde termina el algoritmo. Solo hay UN inicio y UN fin.



Entrada de datos
Pedimos información al usuario o leemos un dato. Verbo: «Leer».



Proceso
Una operación, cálculo o asignación. La acción central del algoritmo.



Decisión
Una pregunta con dos respuestas: Sí / No. El flujo se divide.



Salida / Imprimir
Mostramos un resultado al usuario. Verbo: «Escribir» o «Imprimir».



Flujo de datos
Flechas que conectan los símbolos. Indican el **orden** en que se ejecutan.



Conector
Une partes del diagrama cuando se sale de la página. Se identifica con una letra.

RECUERDA

Cada figura = un tipo de paso

No mezcles formas. Un proceso siempre es un rectángulo. Una decisión siempre es un rombo. Mantener el estándar hace que cualquiera lea tu diagrama.

Los operadores que vamos a usar

Símbolos que nos permiten **calcular** y **comparar** valores. Son los mismos que viste en matemáticas, casi todos.

ARITMÉTICOS

Hacen cálculos

SÍMBOLO	OPERACIÓN	EJEMPLO
+	Suma	$R = A + B$
-	Resta	$R = A - B$
*	Multiplicación	$R = A * B$
/	División	$R = A / B$
^	Potencia	$R = A ^ 2$

COMPARACIÓN (RELACIONALES)

Comparan dos valores

SÍMBOLO	SIGNIFICADO	SE LEE
>	Mayor que	$A > B \rightarrow \text{¿A es mayor?}$
<	Menor que	$A < B \rightarrow \text{¿A es menor?}$
≥	Mayor o igual	$A \geq B$
≤	Menor o igual	$A \leq B$
=	Igual que	$A = B \rightarrow \text{¿son iguales?}$
≠	Diferente	$A \neq B \rightarrow \text{¿son distintos?}$

Volumen de un cubo

PROBLEMA

Dadas las medidas **A**, **B** y **C** (largo, ancho y alto) de una caja con forma de cubo, calcular su volumen.

TABLA DE VARIABLES

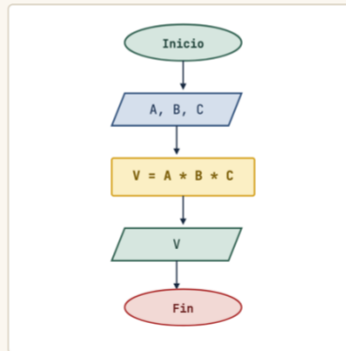
VAR	DESCRIPCIÓN	TIPO
A	Largo del cubo	Real
B	Ancho del cubo	Real
C	Alto del cubo	Real
V	Volumen calculado	Real

PSEUDOCÓDIGO

```

1. Inicio
2. Leer A, B, C
3.  $V = A * B * C$ 
4. Escribir V
5. Fin
  
```

DIAGRAMA DE FLUJO



OBSERVACIÓN

Ambas representaciones **dicen lo mismo**. El diagrama lo cuenta visualmente; el pseudocódigo lo cuenta línea por línea.

Verifica: si $A=2$, $B=3$, $C=4 \rightarrow V = 24$.

ACTIVIDAD GUIADA 2 • 25 MIN

Tu primer diagrama de flujo

En equipos de 2-3 personas, resuelvan los siguientes problemas. Para cada uno: **tabla de variables**, **pseudocódigo** y **diagrama de flujo**.

PROBLEMA 01 • FÁCIL

Conversión de pesos a dólares

Dada una cantidad en pesos y un tipo de cambio, calcular la equivalencia en dólares.

TIP
Identifica primero qué tipo de dato usarás para cada variable.

PROBLEMA 02 • MEDIO

Promedio de tres notas

Pedir tres calificaciones de un estudiante y mostrar el promedio con dos decimales.

TIP
¿Qué tipo de dato usas para el promedio? ¿Entero o real? ¿Por qué?

PROBLEMA 03 • RETO

Suma + producto de dos números

Dados dos números, calcular y mostrar tanto su suma **como** su producto.

TIP
¿Una entrada lleva varias variables? ¿O dos entradas separadas?

❖ **COMPROBACIÓN** Intercambien sus diagramas con otro equipo. ¿Podrían ejecutarlos a mano con datos de prueba?

Lo que nos llevamos

- 1 Identificador = nombre del dato**
Letras descriptivas, sin espacios. Variables (cambian) o constantes (no cambian).
- 2 Cuatro tipos de datos básicos**
Entero · Real · Carácter · Lógico. Cada variable es de UN solo tipo.
- 3 Tabla de variables: paso obligatorio**
Nombre, descripción, tipo, uso (entrada/salida). Siempre antes del algoritmo.
- 4 Siete símbolos estándar**
Inicio · Entrada · Proceso · Decisión · Salida · Flujo · Conector.
- 5 Diagrama y pseudocódigo dicen lo mismo**
Son dos representaciones equivalentes. Practica ambas.

PRÓXIMA SESIÓN

Pseudocódigo y estructuras secuenciales

Aprenderemos las reglas del pseudocódigo y resolveremos varios problemas con la estructura más simple: la **secuencial** (un paso tras otro, sin decisiones).

PROGRESO DEL CURSO

6 h / 15 h · Módulo 2 de 5 completado